

ELEC 374 Digital Systems Engineering

CPU Design Project Phase 3 Report

Author names and student numbers:

Russell Litwin (20398808), Jake Hoffman (20398257), Josh Ossip (20398317)

Lab Group: 26

Submission date: March 30, 2026

Course term: Winter 2026

Academic Integrity Statement

"We do hereby verify that this written lab report is our own work and contains our own original ideas, concepts, and designs. No portion of this report has been copied in whole or in part from another source, with the possible exception of properly referenced material".

1. Objectives and Scope

The goal of this phase was to design and integrate a control unit with the processor datapath developed in Phases 1 and 2, and to verify that the combined system correctly fetches, decodes, and executes all required instructions through functional simulation. In the previous phases the control signals were driven manually by the testbench. In Phase 3 the control unit generates all signals automatically based on the current instruction and execution state.

A test program was loaded into memory at initialization time via hex file. The control unit was expected to begin at the reset state, step through the fetch decode execute cycle for each instruction, and halt when the halt instruction was reached. The final values of all registers and selected memory locations were compared against precomputed expected values to confirm correct behavior.

This report describes the control unit design, the test program, the simulation results, and the waveform evidence for key instruction categories required by the marking rubric.

2. Control Unit Design

The control unit was implemented as a single finite state machine in Verilog, following the Method 1 approach described in the Phase 3 handout. The FSM contains 43 states that cover the full instruction set of the Mini SRC processor. On each clock edge the state

register advances, and a combinational block drives the datapath control signals based on the current state and the opcode field of the instruction register.

The top level integration module `cpu_phase3.v` connects the control unit to the datapath. The control unit reads the instruction register and the branch condition flag from the datapath, and drives all bus selection, register load enable, ALU operation, and memory read write signals back into the datapath. The UseRAM input is tied high because in Phase 3 all memory access goes through the on chip RAM, and the Mdatain port is tied to zero because there is no external memory data source.

The following instruction groups are handled by dedicated state sequences in the control unit:

- Fetch, decode, and halt states shared by all instructions
- R type ALU operations: add, sub, and, or, shr, shra, shl, ror, rol
- Unary operations: neg and not
- Multiply and divide with writeback to HI and LO registers
- Load, load immediate, and store with direct and indexed addressing
- Immediate ALU operations: addi, andi, ori
- Conditional branch instructions: brzr, brnz, brpl, brmi
- Jump and jump and link: jr and jal
- Input output and special register moves: in, out, mfhi, mflo
- No operation

The complete encoding of every instruction in the test program is shown in the table below. Each row gives the memory address, the hexadecimal instruction register value, the five bit opcode, the instruction mnemonic, and the assembly level interpretation. This table can be cross referenced with the waveform evidence to confirm that the IR contents comply with the CPU specification for every instruction executed.

Addr	IR (Hex)	Opcode[31:27]	Instruction	Assembly
0x000	8A800043	10001	ldi	ldi R5, 0x43
0x001	8AA80006	10001	ldi	ldi R5, 0x6(R5)
0x002	82000089	10000	ld	ld R4, 0x89
0x003	8A200004	10001	ldi	ldi R4, 0x4(R4)
0x004	8027FFF8	10000	ld	ld R0, 0x7FFF8(R4)
0x005	89000004	10001	ldi	ldi R2, 0x4
0x006	8A800087	10001	ldi	ldi R5, 0x87
0x007	AA980003	10101	br	brmi R5, 0x3
0x008	8AA80005	10001	ldi	ldi R5, 0x5(R5)
0x009	80AFFFFD	10000	ld	ld R1, 0x7FFFD(R5)
0x00A	D0000000	11010	nop	nop
0x00B	A8900002	10101	br	brpl R1, 0x2
0x00C	89A80007	10001	ldi	ldi R3, 0x7(R5)
0x00D	8B9FFFFC	10001	ldi	ldi R7, 0x7FFFC(R3)
0x00E	03A90000	00000	add	add R7, R5, R2
0x00F	48880003	01001	addi	addi R1, R1, 0x00003
0x010	70880000	01110	neg	neg R1, R1
0x011	78880000	01111	not	not R1, R1
0x012	5088000F	01010	andi	andi R1, R1, 0x0000F
0x013	3A010000	00111	ror	ror R4, R0, R2
0x014	58A00005	01011	ori	ori R1, R4, 0x00005

0x015	2A090000	00101	shra	shra R4, R1, R2
0x016	22A90000	00100	shr	shr R5, R5, R2
0x017	928000A3	10010	st	st 0xA3, R5
0x018	42810000	01000	rol	rol R5, R0, R2
0x019	1B900000	00011	or	or R7, R2, R0
0x01A	12280000	00010	and	and R4, R5, R0
0x01B	93A00089	10010	st	st 0x89(R4), R7
0x01C	082B8000	00001	sub	sub R0, R5, R7
0x01D	32290000	00110	shl	shl R4, R5, R2
0x01E	8B800007	10001	ldi	ldi R7, 0x7
0x01F	89800019	10001	ldi	ldi R3, 0x19
0x020	69B80000	01101	mul	mul R3, R7
0x021	C0800000	11000	mfhi	mfhi R1
0x022	C0000000	11001	mflo	mflo R6
0x023	61B80000	01100	div	div R3, R7
0x024	8C380002	10001	ldi	ldi R8, 0x2(R7)
0x025	8C9FFFFC	10001	ldi	ldi R9, 0x7FFFC(R3)
0x026	8D300003	10001	ldi	ldi R10, 0x3(R6)
0x027	8D880005	10001	ldi	ldi R11, 0x5(R1)
0x028	9D000000	10011	jal	jal R10
0x029	D8000000	11011	halt	halt
0x0B2	07450000	00000	add	add R14, R8, R10
0x0B3	0ECD8000	00001	sub	sub R13, R9, R11
0x0B4	0F768000	00001	sub	sub R14, R14, R13
0x0B5	A6000000	10100	jr	jr R12

3. Test Program and Expected Results

The test program was loaded from the file phase3_mem_init.hex. It contains 42 instructions starting at address zero, a four instruction subroutine at address hexadecimal B2, and two pre initialized data values at addresses hexadecimal 89 and A3. The program exercises every instruction type supported by the control unit, including load and store with both direct and indexed addressing, all arithmetic and logical operations, shifts, rotates, multiply, divide, conditional branches in both taken and not taken directions, jump and link with subroutine return, and the halt instruction.

Two memory locations are initialized before the program runs. Address hexadecimal 89 is loaded with hexadecimal A7, and address hexadecimal A3 is loaded with hexadecimal 68. The program modifies both of these locations during execution through store instructions.

The testbench checks 27 conditions after the program reaches the halt state. These include the final value of every general purpose register R0 through R15, the HI and LO registers, the two memory locations, and several control flow observations confirming that branch taken, branch not taken, jump and link, and jump register paths were all exercised. The transcript output below shows that all 27 checks passed and the program halted in 310 clock cycles.

```

# PASS: halted in 310 cycles
# PASS: halt IR latched value=d8000000
# PASS: R0 final value=00000614
# PASS: R1 final value=00000000
# PASS: R2 final value=00000004
# PASS: R3 final value=00000019
# PASS: R4 final value=00006800
# PASS: R5 final value=00000680
# PASS: R6 final value=000000af
# PASS: R7 final value=00000007
# PASS: R8 final value=00000009
# PASS: R9 final value=00000015
# PASS: R10 final value=000000b2
# PASS: R11 final value=00000005
# PASS: R12 return address value=00000029
# PASS: R13 final value=00000010
# PASS: R14 final value=000000ab
# PASS: R15 final value=00000000
# PASS: HI final value=00000004
# PASS: LO final value=00000003
# PASS: mem[0xA3] final value=00000008
# PASS: mem[0x89] final value=0000006c
# PASS: brmi observed
# PASS: brmi not-taken path observed
# PASS: brpl observed
# PASS: brpl taken path observed (skipped instructions not executed)
# PASS: jal/jr control-flow observed
# PASS PHASE3 CPU SUITE

```

Figure 1. ModelSim transcript output showing all 27 verification checks passing after 310 cycles.

The expected final architectural state is summarized in the following table. All values were confirmed by the testbench.

Register	Final Value	Register	Final Value
R0	0x00000614	R8	0x00000009
R1	0x00000000	R9	0x00000015
R2	0x00000004	R10	0x000000B2
R3	0x00000019	R11	0x00000005
R4	0x00006800	R12	0x00000029
R5	0x00000680	R13	0x00000010
R6	0x000000AF	R14	0x000000AB
R7	0x00000007	R15	0x00000000
HI	0x00000004	LO	0x00000003
mem[0xA3]	0x00000008	mem[0x89]	0x0000006C

4. Functional Simulation Results

All simulation was performed in ModelSim using functional simulation with a ten nanosecond clock period. The waveform display includes all sixteen general purpose registers, the special registers IR, PC, HI, LO, MAR, MDR, Y, and Z, the bus value and bus select, the ALU operation code, the condition logic signals, the select and encode outputs, and all individual control signals from the control unit. The figures below present the waveform evidence organized by rubric category.

Before the program begins executing, all registers are in their reset state. The figure below shows the waveform at time zero, confirming that R0 through R15, HI, LO, MAR, MDR, and all other datapath registers hold the value zero. This establishes the before state required for comparison with the final values.

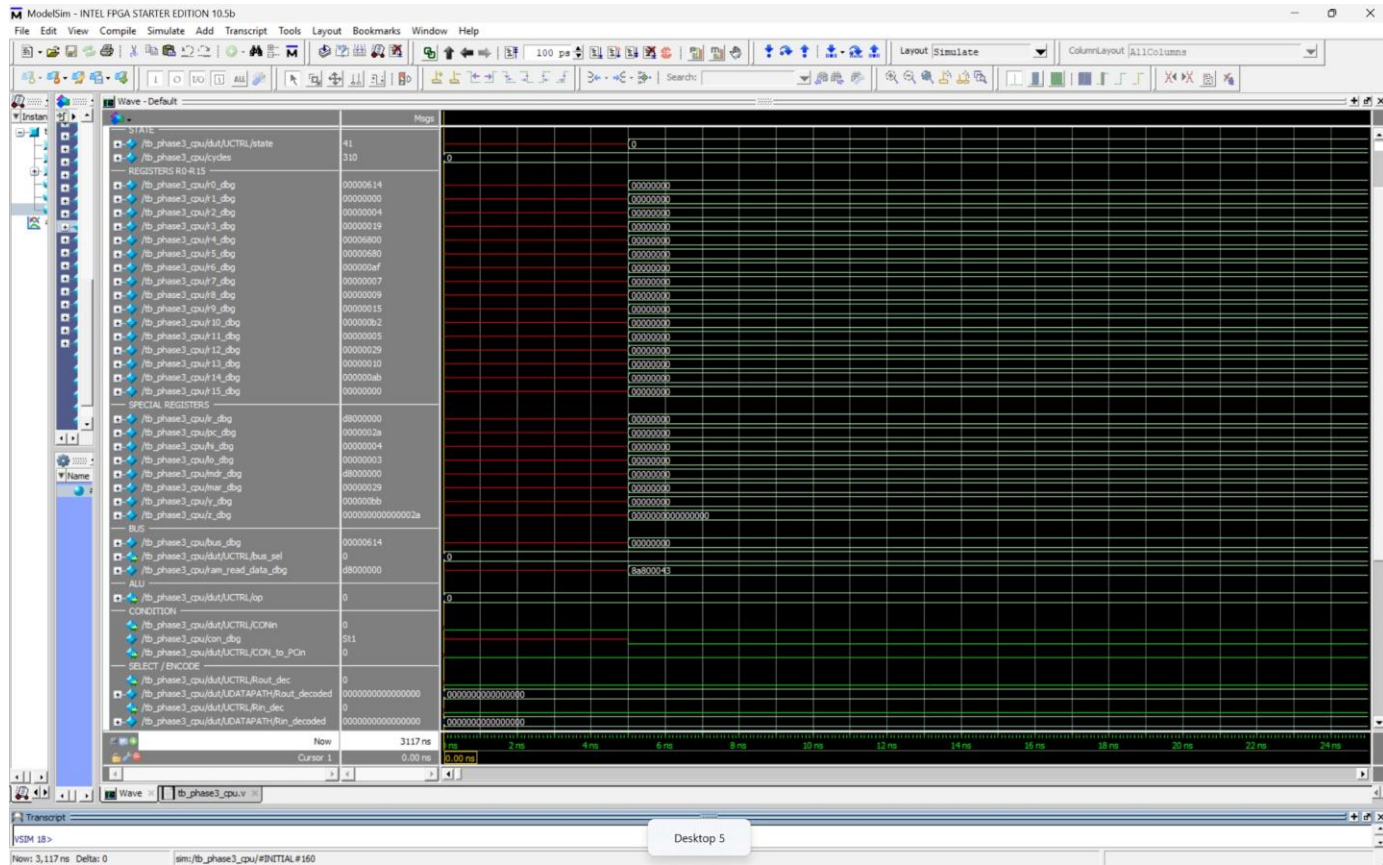


Figure 2. Initial register state at time zero. All registers hold hexadecimal zero.

The instruction fetch cycle occupies three clock states before every instruction. In the first state the program counter is placed on the bus and loaded into the memory address register, while the Z register captures the incremented program counter value. In the second state the incremented value is written back to the program counter, and the memory data register receives the instruction word from RAM. In the third state the memory data register contents are transferred into the instruction register. The figure below shows this sequence for the first instruction fetch, with the state register advancing through values 1, 2, and 3, followed by the decode state 42.

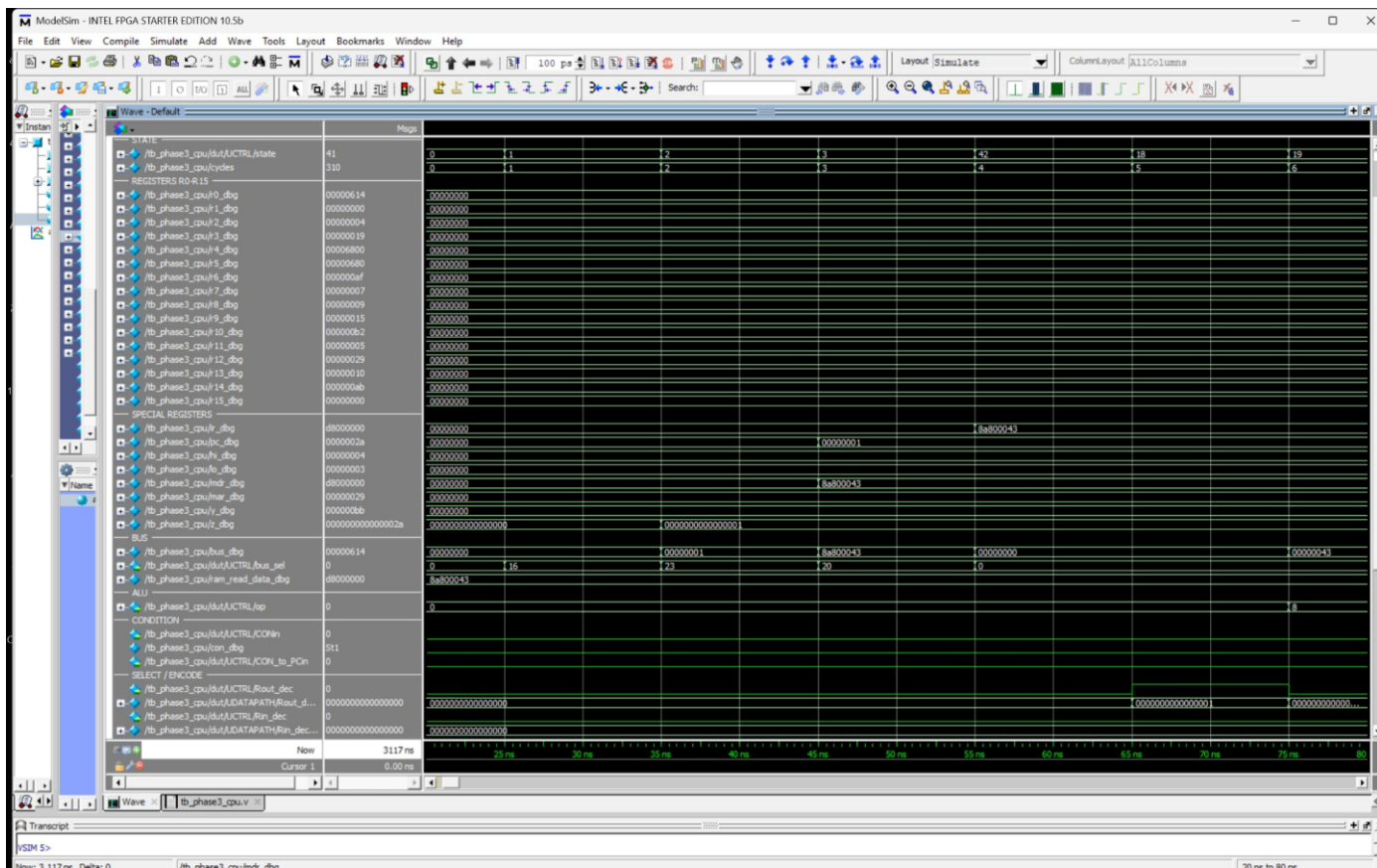


Figure 3. Fetch cycle for the first instruction. State advances 1, 2, 3, 42. PC increments from 0 to 1. IR loads hexadecimal 8A800043.

The brmi instruction at address hexadecimal 07 tests whether R5 is negative. At this point in the program R5 holds a positive value, so the branch condition evaluates to false. The waveform shows the branch execution states 29 through 32. The CONin signal is asserted in state 29 to latch the condition result, and the condition flag remains zero, indicating not taken. The program counter is not updated by the conditional write in state 32, and execution continues with the next sequential instruction at address hexadecimal 08.

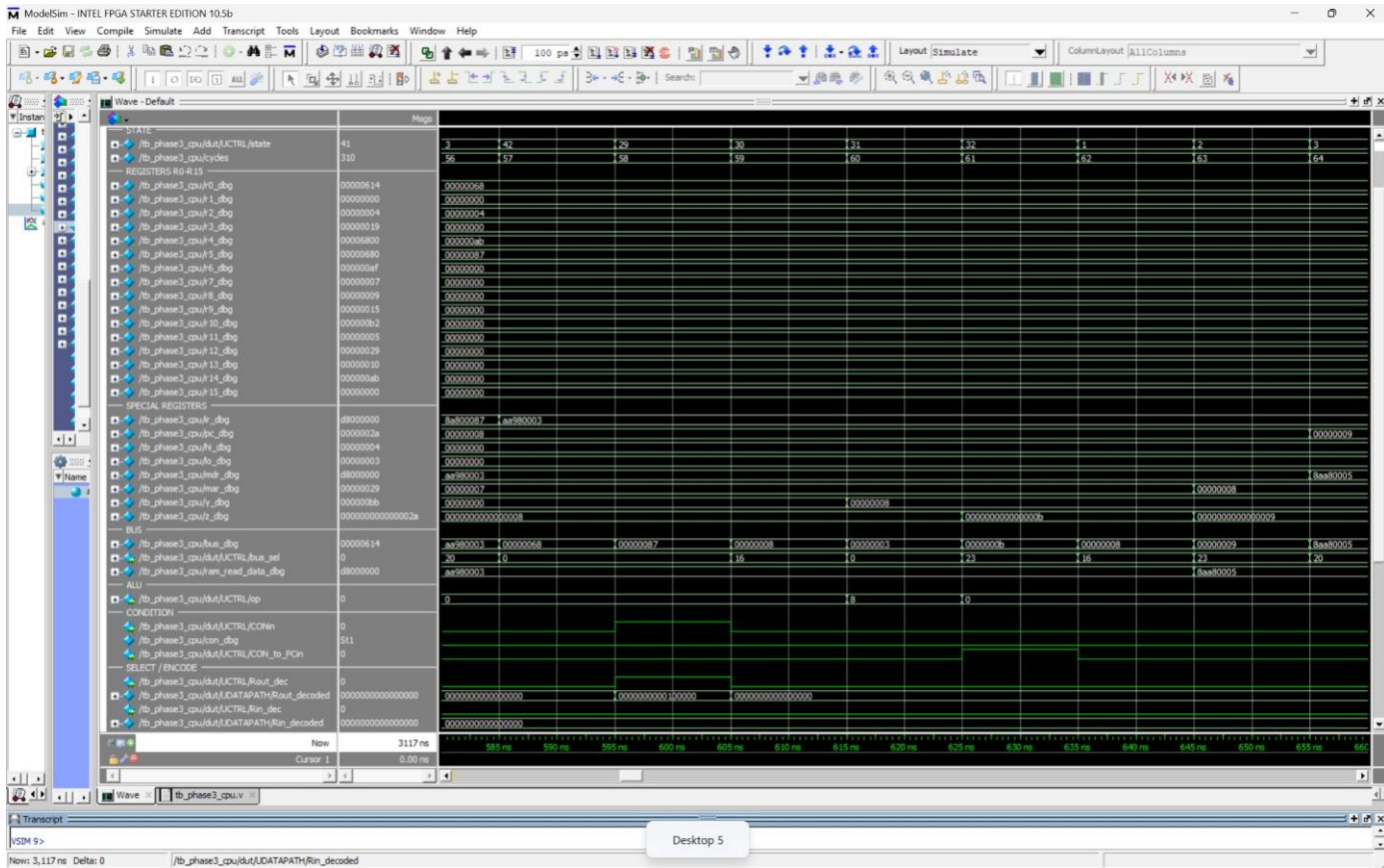


Figure 4. brmi not taken. IR holds AA980003. States 29 through 32 execute. CON flag is zero. PC remains unchanged and the next instruction at address 08 is fetched.

The brpl instruction at address hexadecimal 0B tests whether R1 is positive. R1 holds a nonzero positive value at this point, so the branch condition evaluates to true. The waveform shows the same branch state sequence 29 through 32, but this time the condition flag is set to one. In state 32 the conditional program counter write is enabled and the target address hexadecimal 0E is loaded into the program counter, skipping two instructions at addresses 0C and 0D. The next instruction fetched is the add at address 0E, confirming that the skipped instructions were not executed.

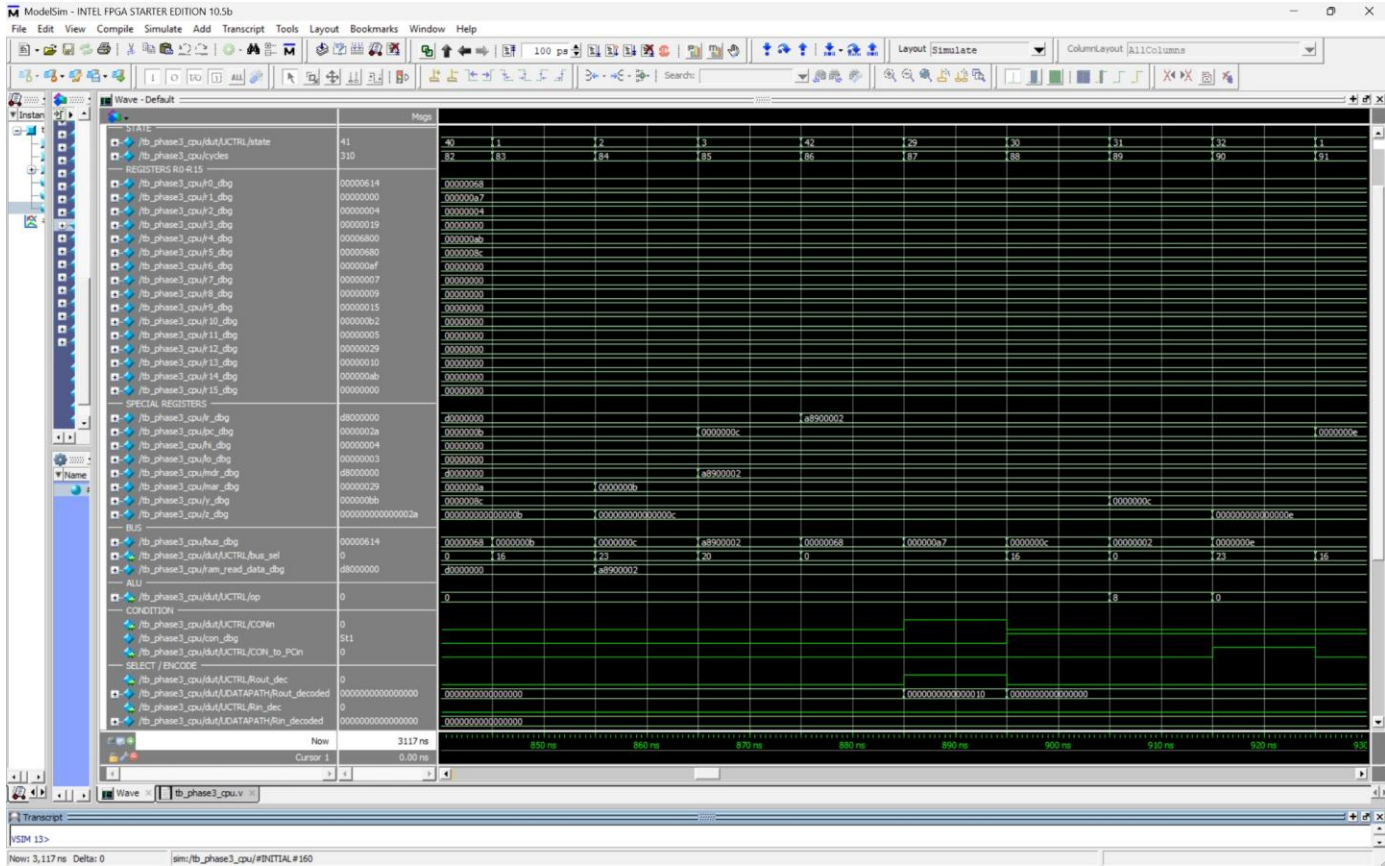


Figure 5. brpl taken. IR holds A8900002. CON flag is one. PC jumps from 0C to 0E, skipping two instructions.

The jal instruction at address hexadecimal 28 performs two operations in states 34 and 35. In state 34 the current program counter value is placed on the bus and written into R12 to save the return address. In state 35 the value in R10, which holds the subroutine entry address hexadecimal B2, is placed on the bus and written into the program counter. The waveform confirms that R12 captures the value hexadecimal 29 and the program counter jumps to hexadecimal B2.

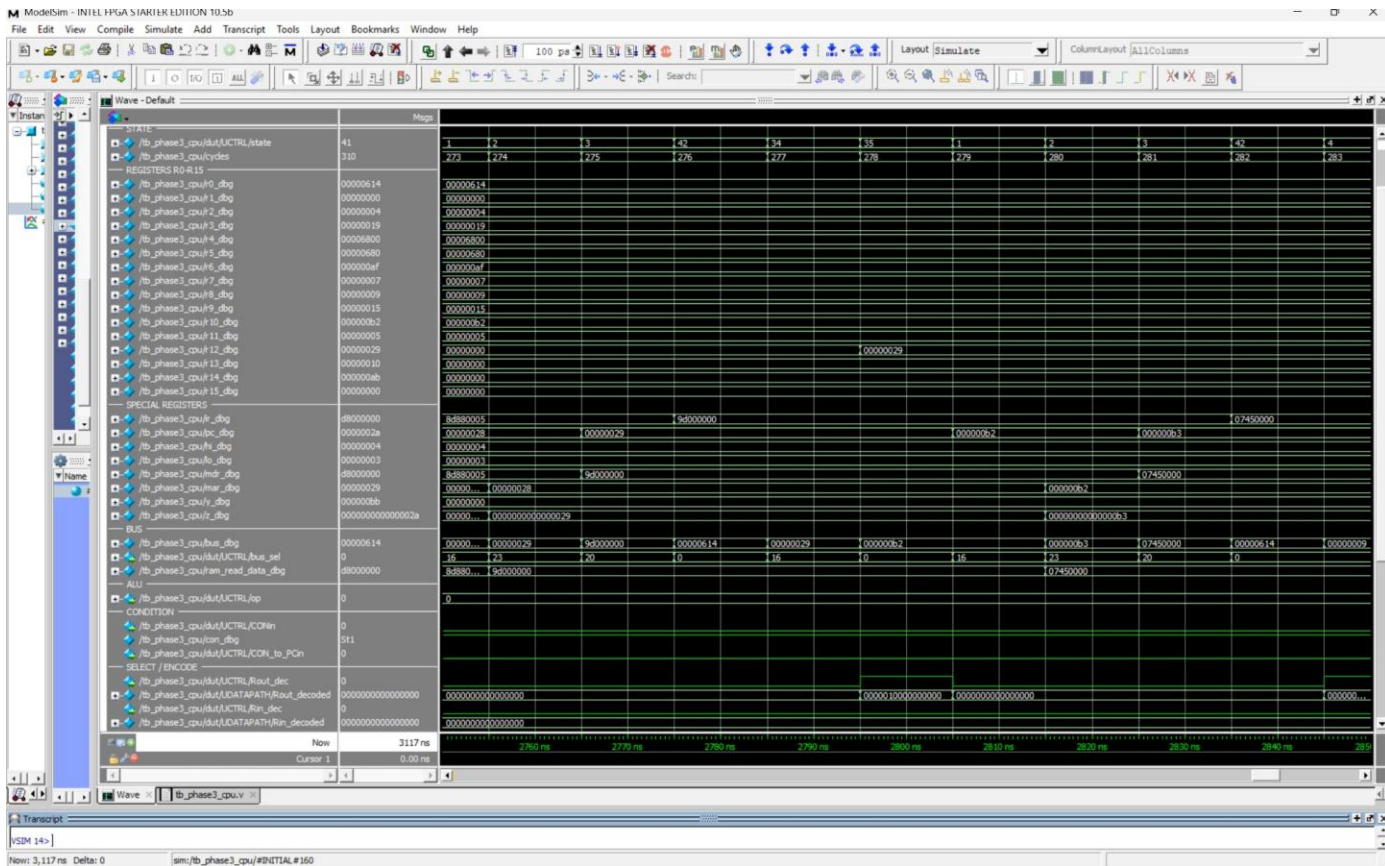


Figure 6a. jal instruction. R12 is loaded with return address hexadecimal 29. PC jumps to subroutine at hexadecimal B2.

The subroutine at address B2 executes three arithmetic instructions before reaching the jr instruction at address B5. The jr instruction places the value of R12 on the bus and writes it into the program counter, returning execution to address hexadecimal 29. The next instruction fetched at that address is the halt instruction, which causes the control unit to enter the halt state and deassert the run signal.

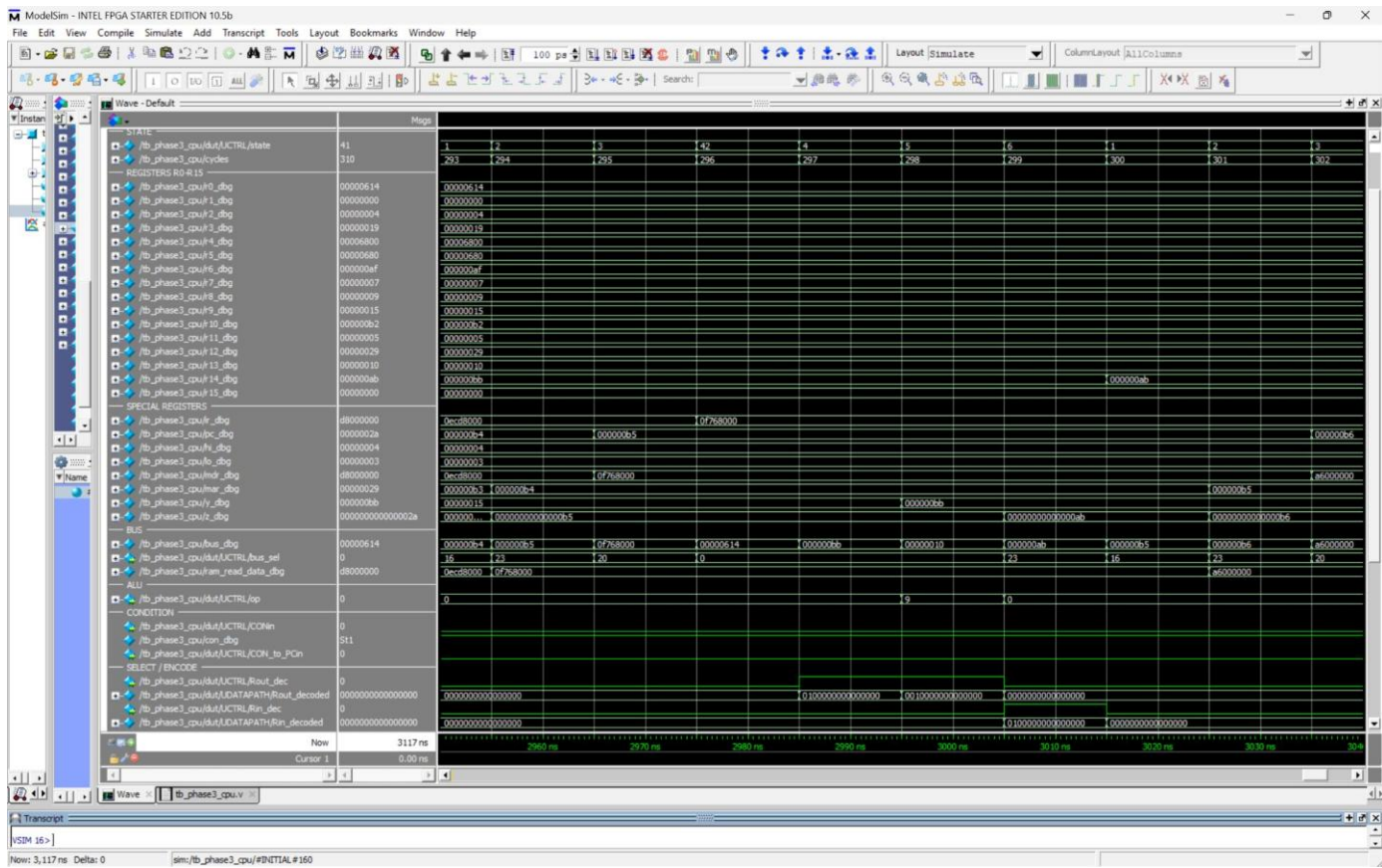


Figure 6b. Subroutine execution at addresses B2 through B4.

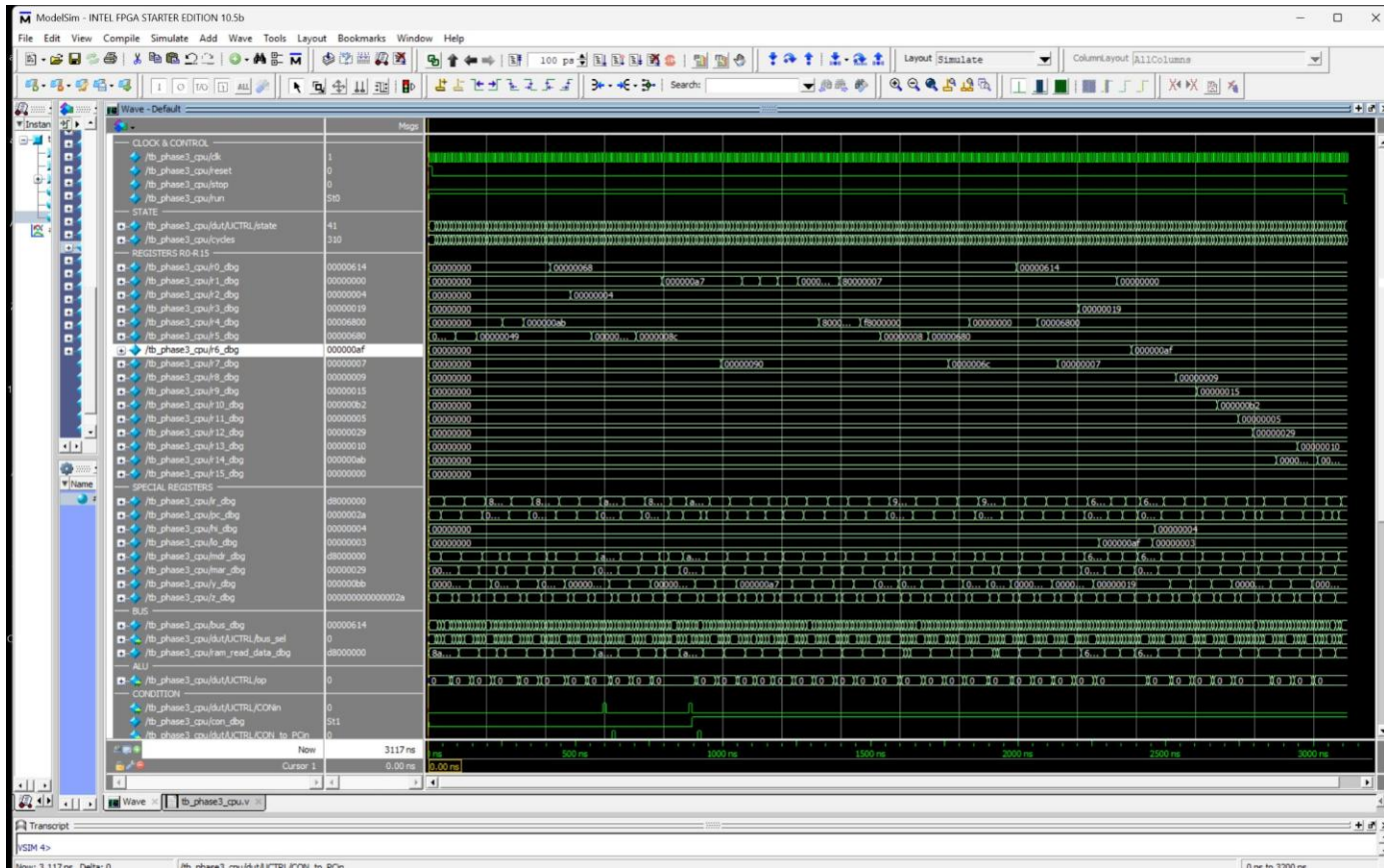


Figure 7. Complete simulation waveform. Final register values are visible in the signal list on the left.

5. Memory Contents Before and After

The test program modifies two memory locations during execution. The table below shows the initial values loaded from the hex file and the final values after the program completes. The store instruction at address hexadecimal 17 writes to memory address A3, and the store instruction at address hexadecimal 1B writes to memory address 89. The MAR and MDR registers reflect the last memory access before halt, which is the fetch of the halt instruction itself. The after values can be confirmed in the section 3 screenshot of the output after the testbench runs.

Element	Before (t = 0)	After (t = 3117 ns)
mem[0x89]	0x000000A7	0x0000006C
mem[0xA3]	0x00000068	0x00000008
MAR	0x00000000	0x00000029
MDR	0x00000000	0xD8000000

The figure below shows the waveform during the store instruction that writes to memory address A3. The MAR register is loaded with the target address, the MDR register holds the data value, and the Write signal is asserted in the final state of the store sequence.

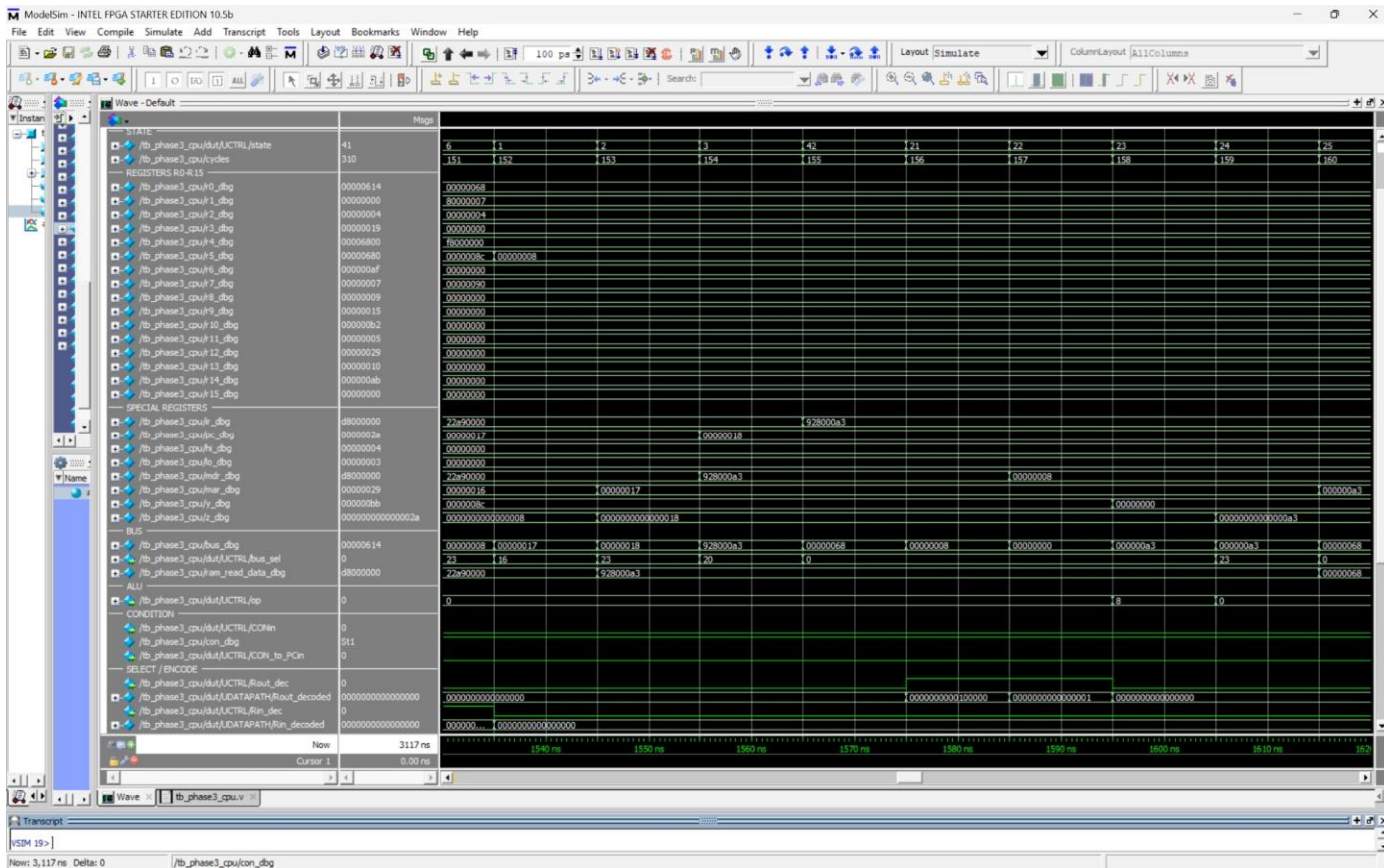


Figure 8. Store instruction writing to memory address A3. MAR and MDR values and Write assertion are visible.

6. Conclusion

Phase 3 functional simulation objectives were met. The control unit correctly generates the control sequence for all supported instructions, including arithmetic and logical operations, load and store, conditional branches in both taken and not taken directions, jump and link with subroutine return, multiply and divide with HI and LO register writeback, and input output operations. The test program ran to completion in 310 clock cycles and all 27 testbench verification checks passed. The waveform evidence confirms correct instruction register contents, proper program counter incrementing and branch behavior, correct final register values, and correct memory contents before and after execution.

Appendix A: HDL Source Code

The following source files constitute the Phase 3 design. The control unit and CPU integration module are new to this phase. The datapath and supporting modules were developed in Phases 1 and 2.

A.1 Control Unit (datapath/control_unit.v)

```
`include "defines.vh"

module control_unit (
    input  wire      clk,
    input  wire      reset,
    input  wire      stop,
    input  wire [31:0] IR,
    input  wire      CON_q,

    output wire      Run,

    output reg [4:0] bus_sel,
    output reg [15:0] Rin,
    output reg      Yin,
    output reg      Zin,
    output reg      PCin,
    output reg      IRin,
    output reg      MARin,
    output reg      MDRin,
    output reg      HIin,
    output reg      LOin,
    output reg      IncPC,
    output reg      Read,
    output reg      Write,

    output reg      Gra,
    output reg      Grb,
    output reg      Grc,
    output reg      Rin_dec,
    output reg      Rout_dec,
    output reg      BAout,
    output reg      Cout,
    output reg      CONin,
    output reg      CON_to_PCin,
    output reg      OutPortin,
    output reg      InPortout,
    output reg      InPortLoad,

    output reg [3:0] op
);

localparam [4:0] SEL_PC    = 5'd16;
localparam [4:0] SEL_MDR  = 5'd20;
localparam [4:0] SEL_HI   = 5'd21;
localparam [4:0] SEL_LO   = 5'd22;
localparam [4:0] SEL_ZLOW = 5'd23;
localparam [4:0] SEL_ZHIGH = 5'd24;

localparam [5:0]
    S_RESET    = 6'd0,
    S_FETCH0   = 6'd1,
    S_FETCH1   = 6'd2,
    S_FETCH2   = 6'd3,
    S_DECODE   = 6'd42,
    S_ALU_T3   = 6'd4,
    S_ALU_T4   = 6'd5,
    S_ALU_T5   = 6'd6,
    S_UNARY_T3 = 6'd7,
```

```

S_UNARY_T4 = 6'd8,
S_MD_T3    = 6'd9,
S_MD_T4    = 6'd10,
S_MD_T5    = 6'd11,
S_MD_T6    = 6'd12,
S_LD_T3    = 6'd13,
S_LD_T4    = 6'd14,
S_LD_T5    = 6'd15,
S_LD_T6    = 6'd16,
S_LD_T7    = 6'd17,
S_LDI_T3   = 6'd18,
S_LDI_T4   = 6'd19,
S_LDI_T5   = 6'd20,
S_ST_T3    = 6'd21,
S_ST_T4    = 6'd22,
S_ST_T5    = 6'd23,
S_ST_T6    = 6'd24,
S_ST_T7    = 6'd25,
S_IMM_T3   = 6'd26,
S_IMM_T4   = 6'd27,
S_IMM_T5   = 6'd28,
S_BR_T3    = 6'd29,
S_BR_T4    = 6'd30,
S_BR_T5    = 6'd31,
S_BR_T6    = 6'd32,
S_JR_T3    = 6'd33,
S_JAL_T3   = 6'd34,
S_JAL_T4   = 6'd35,
S_MFHILO_T3 = 6'd36,
S_OUT_T3   = 6'd37,
S_IN_T3    = 6'd38,
S_IN_T4    = 6'd39,
S_NOP_T3   = 6'd40,
S_HALT     = 6'd41;

reg [5:0] state;
reg [5:0] next_state;
reg      run_q;
reg      next_run;

wire [4:0] opcode = IR[31:27];

assign Run = run_q;

function [3:0] alu_from_opcode;
input [4:0] opcode_f;
begin
    case (opcode_f)
        `OP_ADD: alu_from_opcode = `ADDop;
        `OP_SUB: alu_from_opcode = `SUBop;
        `OP_AND: alu_from_opcode = `ANDop;
        `OP_OR:  alu_from_opcode = `ORop;
        `OP_SHR: alu_from_opcode = `SHRop;
        `OP_SHRA: alu_from_opcode = `SHRAop;
        `OP_SHL: alu_from_opcode = `SHLop;
        `OP_ROR: alu_from_opcode = `RORop;
        `OP_ROL: alu_from_opcode = `ROLop;
        `OP_ADDI: alu_from_opcode = `ADDop;
        `OP_ANDI: alu_from_opcode = `ANDop;
        `OP_ORI:  alu_from_opcode = `ORop;
        `OP_MUL:  alu_from_opcode = `MULop;
        `OP_DIV:  alu_from_opcode = `DIVop;
        `OP_NEG:  alu_from_opcode = `NEGop;
        `OP_NOT:  alu_from_opcode = `NOTop;
        default: alu_from_opcode = 4'd0;
    endcase
end
endfunction

```

```

always @(posedge clk) begin
    if (reset) begin
        state <= S_RESET;
        run_q <= 1'b1;
    end else begin
        state <= next_state;
        run_q <= next_run;
    end
end

always @(*) begin
    next_state = state;
    next_run   = run_q;

    bus_sel    = 5'd0;
    Rin        = 16'd0;
    Yin        = 1'b0;
    Zin        = 1'b0;
    PCin       = 1'b0;
    IRin       = 1'b0;
    MARin      = 1'b0;
    MDRin      = 1'b0;
    HIin       = 1'b0;
    LOin       = 1'b0;
    IncPC      = 1'b0;
    Read       = 1'b0;
    Write      = 1'b0;
    Gra        = 1'b0;
    Grb        = 1'b0;
    Grc        = 1'b0;
    Rin_dec    = 1'b0;
    Rout_dec   = 1'b0;
    BAout      = 1'b0;
    Cout       = 1'b0;
    CONin      = 1'b0;
    CON_to_PCin = 1'b0;
    OutPortin  = 1'b0;
    InPortout  = 1'b0;
    InPortLoad = 1'b0;
    op         = 4'd0;

    if (stop) begin
        next_state = S_HALT;
        next_run   = 1'b0;
    end else begin
        case (state)
            S_RESET: begin
                next_state = S_FETCH0;
                next_run   = 1'b1;
            end

            S_FETCH0: begin
                bus_sel    = SEL_PC;
                MARin      = 1'b1;
                IncPC      = 1'b1;
                Zin        = 1'b1;
                next_state = S_FETCH1;
            end

            S_FETCH1: begin
                bus_sel    = SEL_ZLOW;
                PCin       = 1'b1;
                Read       = 1'b1;
                MDRin      = 1'b1;
                next_state = S_FETCH2;
            end
        endcase
    end
end

```

```

S_FETCH2: begin
    bus_sel    = SEL_MDR;
    IRin       = 1'b1;
    next_state = S_DECODE;
end

S_DECODE: begin
    case (opcode)
        `OP_ADD,
        `OP_SUB,
        `OP_AND,
        `OP_OR,
        `OP_SHR,
        `OP_SHRA,
        `OP_SHL,
        `OP_ROR,
        `OP_ROL: next_state = S_ALU_T3;

        `OP_NEG,
        `OP_NOT: next_state = S_UNARY_T3;

        `OP_MUL,
        `OP_DIV: next_state = S_MD_T3;

        `OP_LD:  next_state = S_LD_T3;
        `OP_LDI: next_state = S_LDI_T3;
        `OP_ST:  next_state = S_ST_T3;

        `OP_ADDI,
        `OP_ANDI,
        `OP_ORI: next_state = S_IMM_T3;

        `OP_BR:  next_state = S_BR_T3;
        `OP_JR:  next_state = S_JR_T3;
        `OP_JAL: next_state = S_JAL_T3;

        `OP_MFHI,
        `OP_MFLO: next_state = S_MFHILO_T3;

        `OP_OUT: next_state = S_OUT_T3;
        `OP_IN:  next_state = S_IN_T3;

        `OP_NOP: next_state = S_NOP_T3;

        `OP_HALT: begin
            next_state = S_HALT;
            next_run    = 1'b0;
        end

        default: begin
            next_state = S_HALT;
            next_run    = 1'b0;
        end
    endcase
end

S_ALU_T3: begin
    Grb    = 1'b1;
    Rout_dec = 1'b1;
    Yin     = 1'b1;
    next_state = S_ALU_T4;
end

S_ALU_T4: begin
    Grc    = 1'b1;
    Rout_dec = 1'b1;
    op      = alu_from_opcode(opcode);
    Zin     = 1'b1;

```

```

        next_state = S_ALU_T5;
    end

    S_ALU_T5: begin
        bus_sel = SEL_ZLOW;
        Gra     = 1'b1;
        Rin_dec = 1'b1;
        next_state = S_FETCH0;
    end

    S_UNARY_T3: begin
        Grb     = 1'b1;
        Rout_dec = 1'b1;
        op      = alu_from_opcode(opcode);
        Zin     = 1'b1;
        next_state = S_UNARY_T4;
    end

    S_UNARY_T4: begin
        bus_sel = SEL_ZLOW;
        Gra     = 1'b1;
        Rin_dec = 1'b1;
        next_state = S_FETCH0;
    end

    S_MD_T3: begin
        Gra     = 1'b1;
        Rout_dec = 1'b1;
        Yin     = 1'b1;
        next_state = S_MD_T4;
    end

    S_MD_T4: begin
        Grb     = 1'b1;
        Rout_dec = 1'b1;
        op      = alu_from_opcode(opcode);
        Zin     = 1'b1;
        next_state = S_MD_T5;
    end

    S_MD_T5: begin
        bus_sel = SEL_ZLOW;
        LOin    = 1'b1;
        next_state = S_MD_T6;
    end

    S_MD_T6: begin
        bus_sel = SEL_ZHIGH;
        HIin    = 1'b1;
        next_state = S_FETCH0;
    end

    S_LD_T3: begin
        Grb     = 1'b1;
        Rout_dec = 1'b1;
        BAout   = 1'b1;
        Yin     = 1'b1;
        next_state = S_LD_T4;
    end

    S_LD_T4: begin
        Cout    = 1'b1;
        op      = `ADDop;
        Zin     = 1'b1;
        next_state = S_LD_T5;
    end

    S_LD_T5: begin

```

```

        bus_sel = SEL_ZLOW;
        MARin   = 1'b1;
        next_state = S_LD_T6;
    end

    S_LD_T6: begin
        Read     = 1'b1;
        MDRin    = 1'b1;
        next_state = S_LD_T7;
    end

    S_LD_T7: begin
        bus_sel = SEL_MDR;
        Gra     = 1'b1;
        Rin_dec = 1'b1;
        next_state = S_FETCH0;
    end

    S_LDI_T3: begin
        Grb     = 1'b1;
        Rout_dec = 1'b1;
        BAout    = 1'b1;
        Yin     = 1'b1;
        next_state = S_LDI_T4;
    end

    S_LDI_T4: begin
        Cout    = 1'b1;
        op      = `ADDop;
        Zin     = 1'b1;
        next_state = S_LDI_T5;
    end

    S_LDI_T5: begin
        bus_sel = SEL_ZLOW;
        Gra     = 1'b1;
        Rin_dec = 1'b1;
        next_state = S_FETCH0;
    end

    S_ST_T3: begin
        Gra     = 1'b1;
        Rout_dec = 1'b1;
        MDRin    = 1'b1;
        next_state = S_ST_T4;
    end

    S_ST_T4: begin
        Grb     = 1'b1;
        Rout_dec = 1'b1;
        BAout    = 1'b1;
        Yin     = 1'b1;
        next_state = S_ST_T5;
    end

    S_ST_T5: begin
        Cout    = 1'b1;
        op      = `ADDop;
        Zin     = 1'b1;
        next_state = S_ST_T6;
    end

    S_ST_T6: begin
        bus_sel = SEL_ZLOW;
        MARin   = 1'b1;
        next_state = S_ST_T7;
    end
end

```

```

S_ST_T7: begin
    Write      = 1'b1;
    next_state = S_FETCH0;
end

S_IMM_T3: begin
    Grb        = 1'b1;
    Rout_dec   = 1'b1;
    Yin        = 1'b1;
    next_state = S_IMM_T4;
end

S_IMM_T4: begin
    Cout       = 1'b1;
    op         = alu_from_opcode(opcode);
    Zin        = 1'b1;
    next_state = S_IMM_T5;
end

S_IMM_T5: begin
    bus_sel    = SEL_ZLOW;
    Gra        = 1'b1;
    Rin_dec    = 1'b1;
    next_state = S_FETCH0;
end

S_BR_T3: begin
    Gra        = 1'b1;
    Rout_dec   = 1'b1;
    CONin      = 1'b1;
    next_state = S_BR_T4;
end

S_BR_T4: begin
    bus_sel    = SEL_PC;
    Yin        = 1'b1;
    next_state = S_BR_T5;
end

S_BR_T5: begin
    Cout       = 1'b1;
    op         = `ADDop;
    Zin        = 1'b1;
    next_state = S_BR_T6;
end

S_BR_T6: begin
    bus_sel    = SEL_ZLOW;
    CON_to_PCin = 1'b1;
    next_state = S_FETCH0;
end

S_JR_T3: begin
    Gra        = 1'b1;
    Rout_dec   = 1'b1;
    PCin       = 1'b1;
    next_state = S_FETCH0;
end

S_JAL_T3: begin
    bus_sel    = SEL_PC;
    Rin[12]    = 1'b1;
    next_state = S_JAL_T4;
end

S_JAL_T4: begin
    Gra        = 1'b1;
    Rout_dec   = 1'b1;

```

```

        PCin      = 1'b1;
        next_state = S_FETCH0;
    end

    S_MFHILO_T3: begin
        bus_sel    = (opcode == `OP_MFHI) ? SEL_HI : SEL_LO;
        Gra        = 1'b1;
        Rin_dec     = 1'b1;
        next_state = S_FETCH0;
    end

    S_OUT_T3: begin
        Gra        = 1'b1;
        Rout_dec   = 1'b1;
        OutPortin  = 1'b1;
        next_state = S_FETCH0;
    end

    S_IN_T3: begin
        InPortLoad = 1'b1;
        next_state = S_IN_T4;
    end

    S_IN_T4: begin
        InPortout  = 1'b1;
        Gra        = 1'b1;
        Rin_dec     = 1'b1;
        next_state = S_FETCH0;
    end

    S_NOP_T3: begin
        next_state = S_FETCH0;
    end

    S_HALT: begin
        next_state = S_HALT;
        next_run   = 1'b0;
    end

    default: begin
        next_state = S_HALT;
        next_run   = 1'b0;
    end
endcase
end
end
endmodule

```

A.2 CPU Top Level Integration (datapath/cpu_phase3.v)

```

module cpu_phase3 #(
    parameter MEM_INIT_FILE = ""
) (
    input wire      clk,
    input wire      reset,
    input wire      stop,
    input wire [31:0] in_port_data,
    input wire [8:0] mem_dbg_addr,

    output wire      run,
    output wire [31:0] bus_dbg,
    output wire [31:0] ir_dbg,
    output wire [31:0] pc_dbg,
    output wire [31:0] mar_dbg,
    output wire [31:0] mdr_dbg,
    output wire [31:0] hi_dbg,

```

```

output wire [31:0] lo_dbg,
output wire [31:0] y_dbg,
output wire [63:0] z_dbg,
output wire      con_dbg,
output wire [31:0] out_port_dbg,
output wire [31:0] in_port_dbg,
output wire [31:0] ram_read_data_dbg,
output wire [31:0] ram_dbg_data,
output wire [31:0] r0_dbg,
output wire [31:0] r1_dbg,
output wire [31:0] r2_dbg,
output wire [31:0] r3_dbg,
output wire [31:0] r4_dbg,
output wire [31:0] r5_dbg,
output wire [31:0] r6_dbg,
output wire [31:0] r7_dbg,
output wire [31:0] r8_dbg,
output wire [31:0] r9_dbg,
output wire [31:0] r10_dbg,
output wire [31:0] r11_dbg,
output wire [31:0] r12_dbg,
output wire [31:0] r13_dbg,
output wire [31:0] r14_dbg,
output wire [31:0] r15_dbg
);

```

```

wire [4:0] bus_sel;
wire [15:0] Rin;
wire      Yin;
wire      Zin;
wire      PCin;
wire      IRin;
wire      MARin;
wire      MDRin;
wire      HIin;
wire      LOin;
wire      IncPC;
wire      Read;
wire      Write;
wire      Gra;
wire      Grb;
wire      Grc;
wire      Rin_dec;
wire      Rout_dec;
wire      BAout;
wire      Cout;
wire      CONin;
wire      CON_to_PCin;
wire      OutPortin;
wire      InPortout;
wire      InPortLoad;
wire [3:0] op;

```

```

control_unit UCTRL (
    .clk(clk),
    .reset(reset),
    .stop(stop),
    .IR(ir_dbg),
    .CON_q(con_dbg),
    .Run(run),
    .bus_sel(bus_sel),
    .Rin(Rin),
    .Yin(Yin),
    .Zin(Zin),
    .PCin(PCin),
    .IRin(IRin),
    .MARin(MARin),
    .MDRin(MDRin),

```

```

        .HIin(HIin),
        .LOin(LOin),
        .IncPC(IncPC),
        .Read(Read),
        .Write(Write),
        .Gra(Gra),
        .Grb(Grb),
        .Grc(Grc),
        .Rin_dec(Rin_dec),
        .Rout_dec(Rout_dec),
        .BAout(BAout),
        .Cout(Cout),
        .CONin(CONin),
        .CON_to_PCin(CON_to_PCin),
        .OutPortin(OutPortin),
        .InPortout(InPortout),
        .InPortLoad(InPortLoad),
        .op(op)
    );

```

```

datapath_logic #(
    .MEM_INIT_FILE(MEM_INIT_FILE)
) UDATAPATH (
    .clk(clk),
    .reset(reset),

    .bus_sel(bus_sel),
    .Rin(Rin),
    .Yin(Yin),
    .Zin(Zin),

    .PCin(PCin),
    .IRin(IRin),
    .MARin(MARin),
    .MDRin(MDRin),
    .HIin(HIin),
    .LOin(LOin),
    .IncPC(IncPC),
    .Read(Read),
    .Write(Write),
    .Mdatain(32'b0),

    .Gra(Gra),
    .Grb(Grb),
    .Grc(Grc),
    .Rin_dec(Rin_dec),
    .Rout_dec(Rout_dec),
    .BAout(BAout),
    .Cout(Cout),
    .CONin(CONin),
    .CON_to_PCin(CON_to_PCin),
    .OutPortin(OutPortin),
    .InPortout(InPortout),
    .InPortLoad(InPortLoad),
    .InPort_data(in_port_data),
    .UseRAM(1'b1),
    .MemDbgAddr(mem_dbg_addr),

    .PC(32'b0),
    .IR(32'b0),
    .HI(32'b0),
    .LO(32'b0),
    .MAR(32'b0),
    .MDR(32'b0),

    .op(op),

    .BUS(bus_dbg),

```

```

        .R0_q(r0_dbg),
        .R1_q(r1_dbg),
        .R2_q(r2_dbg),
        .R3_q(r3_dbg),
        .R4_q(r4_dbg),
        .R5_q(r5_dbg),
        .R6_q(r6_dbg),
        .R7_q(r7_dbg),
        .R8_q(r8_dbg),
        .R9_q(r9_dbg),
        .R10_q(r10_dbg),
        .R11_q(r11_dbg),
        .R12_q(r12_dbg),
        .R13_q(r13_dbg),
        .R14_q(r14_dbg),
        .R15_q(r15_dbg),
        .HI_q_dbg(hi_dbg),
        .LO_q_dbg(lo_dbg),
        .IR_q_dbg(ir_dbg),
        .PC_q_dbg(pc_dbg),
        .MAR_q_dbg(mar_dbg),
        .MDR_q_dbg(mdr_dbg),
        .Y_q(y_dbg),
        .Z_q(z_dbg),
        .CON_q_dbg(con_dbg),
        .OutPort_q_dbg(out_port_dbg),
        .InPort_q_dbg(in_port_dbg),
        .RAM_read_data_dbg(ram_read_data_dbg),
        .RAM_dbg_data(ram_dbg_data)
    );

endmodule

```

A.3 Datapath (datapath/datapath_logic.v)

```

module datapath_logic #(
    parameter MEM_INIT_FILE = ""
) (
    input  wire      clk,
    input  wire      reset,

    // bus control
    input  wire [4:0] bus_sel,
    input  wire [15:0] Rin,
    input  wire      Yin,
    input  wire      Zin,

    // control of internal special registers
    input  wire      PCin,
    input  wire      IRin,
    input  wire      MARin,
    input  wire      MDRin,
    input  wire      HIin,
    input  wire      LOin,
    input  wire      IncPC,
    input  wire      Read,
    input  wire      Write,
    input  wire [31:0] Mdatain,

    // phase 2 controls
    input  wire      Gra,
    input  wire      Grb,
    input  wire      Grc,
    input  wire      Rin_dec,
    input  wire      Rout_dec,
    input  wire      BAOut,
    input  wire      Cout,

```

```

input  wire      CONin,
input  wire      CON_to_PCin,
input  wire      OutPortin,
input  wire      InPortout,
input  wire      InPortLoad,
input  wire [31:0] InPort_data,
input  wire      UseRAM,
input  wire [8:0] MemDbgAddr,

// reset-time seed values for internal special registers
input  wire [31:0] PC,
input  wire [31:0] IR,
input  wire [31:0] HI,
input  wire [31:0] LO,
input  wire [31:0] MAR,
input  wire [31:0] MDR,

// ALU logic controls
input  wire [3:0] op,

// debug/visibility
output wire [31:0] BUS,
output wire [31:0] R5_q,
output wire [31:0] R6_q,
output wire [31:0] R3_q,
output wire [31:0] R2_q,
output wire [31:0] R0_q,
output wire [31:0] R1_q,
output wire [31:0] R4_q,
output wire [31:0] R12_q,
output wire [31:0] R7_q,
output wire [31:0] R8_q,
output wire [31:0] R9_q,
output wire [31:0] R10_q,
output wire [31:0] R11_q,
output wire [31:0] R13_q,
output wire [31:0] R14_q,
output wire [31:0] R15_q,
output wire [31:0] HI_q_dbg,
output wire [31:0] LO_q_dbg,
output wire [31:0] IR_q_dbg,
output wire [31:0] PC_q_dbg,
output wire [31:0] MAR_q_dbg,
output wire [31:0] MDR_q_dbg,
output wire [31:0] Y_q,
output wire [63:0] Z_q,

// phase 2 debug
output wire      CON_q_dbg,
output wire [31:0] OutPort_q_dbg,
output wire [31:0] InPort_q_dbg,
output wire [31:0] RAM_read_data_dbg,
output wire [31:0] RAM_dbg_data
);

// ----- Registers R0..R15 -----
wire [31:0] R [0:15];
wire [15:0] Rin_decoded;
wire [15:0] Rout_decoded;
wire [3:0]  decoded_reg_idx;
wire [15:0] Rin_effective;

assign Rin_effective = Rin | Rin_decoded;

genvar i;
generate
  for (i = 0; i < 16; i = i + 1) begin : GEN_REGS
    register32 Ri (

```

```

        .clk(clk), .reset(reset),
        .en(Rin_effective[i]),
        .d(BUS),
        .q(R[i])
    );
end
endgenerate

assign R5_q = R[5];
assign R6_q = R[6];
assign R2_q = R[2];
assign R3_q = R[3];
assign R0_q = R[0];
assign R1_q = R[1];
assign R4_q = R[4];
assign R12_q = R[12];
assign R7_q = R[7];
assign R8_q = R[8];
assign R9_q = R[9];
assign R10_q = R[10];
assign R11_q = R[11];
assign R13_q = R[13];
assign R14_q = R[14];
assign R15_q = R[15];

// ----- Internal special registers -----
reg [31:0] PC_q;
reg [31:0] IR_q;
reg [31:0] MAR_q;
reg [31:0] MDR_q;
reg [31:0] HI_q;
reg [31:0] LO_q;
reg [31:0] decoded_rout_data;

wire [31:0] C_sign_extended;
wire        CON_q;
wire [31:0] out_port_q;
wire [31:0] in_port_q;
wire [31:0] ram_read_data;
wire [31:0] bus_mux_out;
wire        decoded_rout_active = |Rout_decoded;
wire        use_ram_data = (UseRAM === 1'b1);
wire        write_mem = (Write === 1'b1);
wire        cout_en = (Cout === 1'b1);
wire        inportout_en = (InPortout === 1'b1);

// ----- Y register (operand A latch) -----
register32 Yreg (
    .clk(clk), .reset(reset),
    .en(Yin),
    .d(BUS),
    .q(Y_q)
);

// ----- ALU logic (A=Y, B=BUS) -----
wire [63:0] ALU_C;

alu_logic ULOGIC (
    .A(Y_q),
    .B(BUS),
    .op(op),
    .overflow(),
    .C(ALU_C)
);

// IncPC path for T0: Z <- PC + 1 when IncPC is asserted.
wire [63:0] Z_d = IncPC ? {32'b0, (BUS + 32'd1)} : ALU_C;

```

```

// ----- Z register -----
register64 Zreg (
    .clk(clk), .reset(reset),
    .en(Zin),
    .d(Z_d),
    .q(Z_q)
);

wire [31:0] Zlow = Z_q[31:0];
wire [31:0] Zhigh = Z_q[63:32];

assign HI_q_dbg = HI_q;
assign LO_q_dbg = LO_q;
assign IR_q_dbg = IR_q;
assign PC_q_dbg = PC_q;
assign MAR_q_dbg = MAR_q;
assign MDR_q_dbg = MDR_q;
assign CON_q_dbg = CON_q;
assign OutPort_q_dbg = out_port_q;
assign InPort_q_dbg = in_port_q;
assign RAM_read_data_dbg = ram_read_data;

select_encode USELECT (
    .IR(IR_q),
    .Gra(Gra),
    .Grb(Grb),
    .Grc(Grc),
    .Rin_sel(Rin_dec),
    .Rout_sel(Rout_dec),
    .Rin_decoded(Rin_decoded),
    .Rout_decoded(Rout_decoded),
    .selected_reg(decoded_reg_idx)
);

sign_extend_c USEXT (
    .IR(IR_q),
    .Cout(Cout),
    .C_sign_extended(C_sign_extended)
);

con_ff_logic UCON (
    .clk(clk),
    .reset(reset),
    .CONin(CONin),
    .bus_value(BUS),
    .c2(IR_q[20:19]),
    .CON_q(CON_q)
);

io_ports UIO (
    .clk(clk),
    .reset(reset),
    .OutPortin(OutPortin),
    .bus_in(BUS),
    .InPortLoad(InPortLoad),
    .InPort_data(InPort_data),
    .OutPort_q(out_port_q),
    .InPort_q(in_port_q)
);

ram512x32 #(
    .MEM_INIT_FILE(MEM_INIT_FILE)
) URAM (
    .clk(clk),
    .write_en(write_mem),
    .addr(MAR_q[8:0]),
    .write_data(MDR_q),
    .read_data(ram_read_data),

```

```

        .dbg_addr(MemDbgAddr),
        .dbg_data(RAM_dbg_data)
    );

always @(*) begin
    case (decoded_reg_idx)
        4'd0: decoded_rout_data = R[0];
        4'd1: decoded_rout_data = R[1];
        4'd2: decoded_rout_data = R[2];
        4'd3: decoded_rout_data = R[3];
        4'd4: decoded_rout_data = R[4];
        4'd5: decoded_rout_data = R[5];
        4'd6: decoded_rout_data = R[6];
        4'd7: decoded_rout_data = R[7];
        4'd8: decoded_rout_data = R[8];
        4'd9: decoded_rout_data = R[9];
        4'd10: decoded_rout_data = R[10];
        4'd11: decoded_rout_data = R[11];
        4'd12: decoded_rout_data = R[12];
        4'd13: decoded_rout_data = R[13];
        4'd14: decoded_rout_data = R[14];
        4'd15: decoded_rout_data = R[15];
        default: decoded_rout_data = 32'b0;
    endcase
end

wire [31:0] decoded_bus_value = (BAout && (decoded_reg_idx == 4'd0)) ? 32'b0 : decoded_rout_data;

assign BUS = decoded_rout_active ? decoded_bus_value :
    (cout_en ? C_sign_extended :
    (inportout_en ? in_port_q : bus_mux_out));

// Special registers load from control signals.
always @(posedge clk) begin
    if (reset) begin
        PC_q <= PC;
        IR_q <= IR;
        MAR_q <= MAR;
        MDR_q <= MDR;
        HI_q <= HI;
        LO_q <= LO;
    end else begin
        if (MARin) begin
            MAR_q <= BUS;
        end

        if (MDRin) begin
            MDR_q <= Read ? (use_ram_data ? ram_read_data : Mdatain) : BUS;
        end

        if (IRin) begin
            IR_q <= BUS;
        end

        if (PCin || ((CON_to_PCin === 1'b1) && CON_q)) begin
            PC_q <= BUS;
        end

        if (HIin) begin
            HI_q <= BUS;
        end

        if (LOin) begin
            LO_q <= BUS;
        end
    end
end

```

```

bus_mux UBUS (
    .sel(bus_sel),

    .R0(R[0]), .R1(R[1]), .R2(R[2]), .R3(R[3]),
    .R4(R[4]), .R5(R[5]), .R6(R[6]), .R7(R[7]),
    .R8(R[8]), .R9(R[9]), .R10(R[10]), .R11(R[11]),
    .R12(R[12]), .R13(R[13]), .R14(R[14]), .R15(R[15]),

    .PC(PC_q),
    .IR(IR_q),
    .Y(Y_q),
    .MAR(MAR_q),
    .MDR(MDR_q),
    .HI(HI_q),
    .LO(LO_q),

    .Zlow(Zlow),
    .Zhigh(Zhigh),

    .bus(bus_mux_out)
);

endmodule

```

A.4 Defines (datapath/defines.vh)

```

`ifndef DEFINES_VH
`define DEFINES_VH

`define ANDop    4'd1
`define ORop     4'd2
`define SHLop    4'd3
`define SHRop    4'd4
`define SHRAop   4'd5
`define ROLop    4'd6
`define RORop    4'd7
`define ADDop    4'd8
`define SUBop    4'd9
`define MULop    4'd10
`define DIVop    4'd11
`define NEGop    4'd12
`define NOTop    4'd13

`define OP_ADD    5'd0
`define OP_SUB    5'd1
`define OP_AND    5'd2
`define OP_OR     5'd3
`define OP_SHR    5'd4
`define OP_SHRA   5'd5
`define OP_SHL    5'd6
`define OP_ROR    5'd7
`define OP_ROL    5'd8
`define OP_ADDI   5'd9
`define OP_ANDI   5'd10
`define OP_ORI    5'd11
`define OP_DIV    5'd12
`define OP_MUL    5'd13
`define OP_NEG    5'd14
`define OP_NOT    5'd15
`define OP_LD     5'd16
`define OP_LDI    5'd17
`define OP_ST     5'd18
`define OP_JAL    5'd19
`define OP_JR     5'd20
`define OP_BR     5'd21
`define OP_IN     5'd22
`define OP_OUT    5'd23

```

```

`define OP_MFHI    5'd24
`define OP_MFLO    5'd25
`define OP_NOP     5'd26
`define OP_HALT    5'd27

`endif

```

Appendix B: Testbench and Test Program

B.1 Phase 3 Testbench (tb/tb_phase3_cpu.v)

```

`timescale 1ns/1ps

module tb_phase3_cpu;
    reg clk = 1'b0;
    always #5 clk = ~clk;

    reg        reset;
    reg        stop;
    reg [31:0] in_port_data;
    reg [8:0]  mem_dbg_addr;

    wire        run;
    wire [31:0] bus_dbg;
    wire [31:0] ir_dbg;
    wire [31:0] pc_dbg;
    wire [31:0] mar_dbg;
    wire [31:0] mdr_dbg;
    wire [31:0] hi_dbg;
    wire [31:0] lo_dbg;
    wire [31:0] y_dbg;
    wire [63:0] z_dbg;
    wire        con_dbg;
    wire [31:0] out_port_dbg;
    wire [31:0] in_port_dbg;
    wire [31:0] ram_read_data_dbg;
    wire [31:0] ram_dbg_data;
    wire [31:0] r0_dbg;
    wire [31:0] r1_dbg;
    wire [31:0] r2_dbg;
    wire [31:0] r3_dbg;
    wire [31:0] r4_dbg;
    wire [31:0] r5_dbg;
    wire [31:0] r6_dbg;
    wire [31:0] r7_dbg;
    wire [31:0] r8_dbg;
    wire [31:0] r9_dbg;
    wire [31:0] r10_dbg;
    wire [31:0] r11_dbg;
    wire [31:0] r12_dbg;
    wire [31:0] r13_dbg;
    wire [31:0] r14_dbg;
    wire [31:0] r15_dbg;

    localparam [31:0] IR_BRMI      = 32'hAA980003;
    localparam [31:0] IR_BRPL      = 32'hA8900002;
    localparam [31:0] IR_LDI_AFTER_BRMI = 32'h8AA80005;
    localparam [31:0] IR_SKIP_12    = 32'h89A80007;
    localparam [31:0] IR_SKIP_13    = 32'h8B9FFFFC;
    localparam [31:0] IR_JAL        = 32'h9D000000;
    localparam [31:0] IR_JR         = 32'hA6000000;
    localparam [31:0] IR_HALT       = 32'hD8000000;

    integer failures;

```

```

integer cycles;

reg seen_brmi;
reg seen_brpl;
reg seen_after_brmi;
reg seen_skip_12;
reg seen_skip_13;
reg seen_jal;
reg seen_jr;
reg seen_jal_effect;

cpu_phase3 #(
    .MEM_INIT_FILE("tb/phase3_mem_init.hex")
) dut (
    .clk(clk),
    .reset(reset),
    .stop(stop),
    .in_port_data(in_port_data),
    .mem_dbg_addr(mem_dbg_addr),
    .run(run),
    .bus_dbg(bus_dbg),
    .ir_dbg(ir_dbg),
    .pc_dbg(pc_dbg),
    .mar_dbg(mar_dbg),
    .mdr_dbg(mdr_dbg),
    .hi_dbg(hi_dbg),
    .lo_dbg(lo_dbg),
    .y_dbg(y_dbg),
    .z_dbg(z_dbg),
    .con_dbg(con_dbg),
    .out_port_dbg(out_port_dbg),
    .in_port_dbg(in_port_dbg),
    .ram_read_data_dbg(ram_read_data_dbg),
    .ram_dbg_data(ram_dbg_data),
    .r0_dbg(r0_dbg),
    .r1_dbg(r1_dbg),
    .r2_dbg(r2_dbg),
    .r3_dbg(r3_dbg),
    .r4_dbg(r4_dbg),
    .r5_dbg(r5_dbg),
    .r6_dbg(r6_dbg),
    .r7_dbg(r7_dbg),
    .r8_dbg(r8_dbg),
    .r9_dbg(r9_dbg),
    .r10_dbg(r10_dbg),
    .r11_dbg(r11_dbg),
    .r12_dbg(r12_dbg),
    .r13_dbg(r13_dbg),
    .r14_dbg(r14_dbg),
    .r15_dbg(r15_dbg)
);

task check_eq;
    input [31:0] got;
    input [31:0] exp;
    input [255:0] msg;
    begin
        if (got != exp) begin
            failures = failures + 1;
            $display("FAIL: %0s got=%h exp=%h", msg, got, exp);
        end else begin
            $display("PASS: %0s value=%h", msg, got);
        end
    end
endtask

task check_mem;
    input [8:0] addr;

```

```

input [31:0] exp;
input [255:0] msg;
begin
    mem_dbg_addr = addr;
    #1;
    check_eq(ram_dbg_data, exp, msg);
end
endtask

always @(posedge clk) begin
    if (!reset) begin
        if (ir_dbg == IR_BRMI) begin
            seen_brmi <= 1'b1;
        end
        if (ir_dbg == IR_BRPL) begin
            seen_brpl <= 1'b1;
        end
        if (ir_dbg == IR_LDI_AFTER_BRMI) begin
            seen_after_brmi <= 1'b1;
        end
        if (ir_dbg == IR_SKIP_12) begin
            seen_skip_12 <= 1'b1;
        end
        if (ir_dbg == IR_SKIP_13) begin
            seen_skip_13 <= 1'b1;
        end
        if (ir_dbg == IR_JAL) begin
            seen_jal <= 1'b1;
        end
        if (ir_dbg == IR_JR) begin
            seen_jr <= 1'b1;
        end
        if ((r12_dbg == 32'h00000029) && (pc_dbg == 32'h000000B2)) begin
            seen_jal_effect <= 1'b1;
        end
    end
end

initial begin
    $dumpfile("phase3_cpu.vcd");
    $dumpvars(0, tb_phase3_cpu);

    failures = 0;
    cycles = 0;

    seen_brmi = 1'b0;
    seen_brpl = 1'b0;
    seen_after_brmi = 1'b0;
    seen_skip_12 = 1'b0;
    seen_skip_13 = 1'b0;
    seen_jal = 1'b0;
    seen_jr = 1'b0;
    seen_jal_effect = 1'b0;

    stop = 1'b0;
    in_port_data = 32'b0;
    mem_dbg_addr = 9'd0;

    reset = 1'b1;
    @(posedge clk);
    @(posedge clk);
    reset = 1'b0;

    while (run && (cycles < 5000)) begin
        @(posedge clk);
        cycles = cycles + 1;
    end
end

```

```

if (run) begin
    failures = failures + 1;
    $display("FAIL: timeout waiting for halt");
end else begin
    $display("PASS: halted in %0d cycles", cycles);
end

// Final architectural state checks.
check_eq(ir_dbg, IR_HALT, "halt IR latched");
check_eq(r0_dbg, 32'h00000614, "R0 final");
check_eq(r1_dbg, 32'h00000000, "R1 final");
check_eq(r2_dbg, 32'h00000004, "R2 final");
check_eq(r3_dbg, 32'h00000019, "R3 final");
check_eq(r4_dbg, 32'h00006800, "R4 final");
check_eq(r5_dbg, 32'h00000680, "R5 final");
check_eq(r6_dbg, 32'h000000AF, "R6 final");
check_eq(r7_dbg, 32'h00000007, "R7 final");
check_eq(r8_dbg, 32'h00000009, "R8 final");
check_eq(r9_dbg, 32'h00000015, "R9 final");
check_eq(r10_dbg, 32'h000000B2, "R10 final");
check_eq(r11_dbg, 32'h00000005, "R11 final");
check_eq(r12_dbg, 32'h00000029, "R12 return address");
check_eq(r13_dbg, 32'h00000010, "R13 final");
check_eq(r14_dbg, 32'h000000AB, "R14 final");
check_eq(r15_dbg, 32'h00000000, "R15 final");
check_eq(hi_dbg, 32'h00000004, "HI final");
check_eq(lo_dbg, 32'h00000003, "LO final");

check_mem(9'h0A3, 32'h00000008, "mem[0xA3] final");
check_mem(9'h089, 32'h0000006C, "mem[0x89] final");

// Control-flow evidence checks.
if (!seen_brmi) begin
    failures = failures + 1;
    $display("FAIL: brmi instruction was not observed");
end else begin
    $display("PASS: brmi observed");
end

if (!seen_after_brmi) begin
    failures = failures + 1;
    $display("FAIL: brmi not-taken path evidence missing (next instruction not observed)");
end else begin
    $display("PASS: brmi not-taken path observed");
end

if (!seen_brpl) begin
    failures = failures + 1;
    $display("FAIL: brpl instruction was not observed");
end else begin
    $display("PASS: brpl observed");
end

if (seen_skip_12 || seen_skip_13) begin
    failures = failures + 1;
    $display("FAIL: brpl-taken path violated (skipped instructions executed)");
end else begin
    $display("PASS: brpl taken path observed (skipped instructions not executed)");
end

if (!seen_jal || !seen_jal_effect || !seen_jr) begin
    failures = failures + 1;
    $display("FAIL: jal/jr control-flow evidence missing jal=%0d jal_effect=%0d jr=%0d",
        seen_jal, seen_jal_effect, seen_jr);
end else begin
    $display("PASS: jal/jr control-flow observed");
end

```

```

        if (failures == 0) begin
            $display("PASS PHASE3 CPU SUITE");
        end else begin
            $display("FAIL PHASE3 CPU SUITE: failures=%0d", failures);
            $fatal;
        end

        $finish;
    end

endmodule

```

B.2 Memory Initialization File (tb/phase3_mem_init.hex)

```

@000
8A800043
8AA80006
82000089
8A200004
8027FFF8
89000004
8A800087
AA980003
8AA80005
80AFFFFD
D0000000
A8900002
89A80007
8B9FFFC
03A90000
48880003
70880000
78880000
5088000F
3A010000
58A00005
2A090000
22A90000
928000A3
42810000
1B900000
12280000
93A00089
082B8000
32290000
8B800007
89800019
69B80000
C0800000
CB000000
61B80000
8C380002
8C9FFFC
8D300003
8D880005
9D000000
D8000000
@089
000000A7
@0A3
00000068
@0B2
07450000
0ECD8000
0F768000

```

A6000000